

6.5830/1 Lecture 5: Access Methods & Indexing

2/19/2026

Tianyu Li

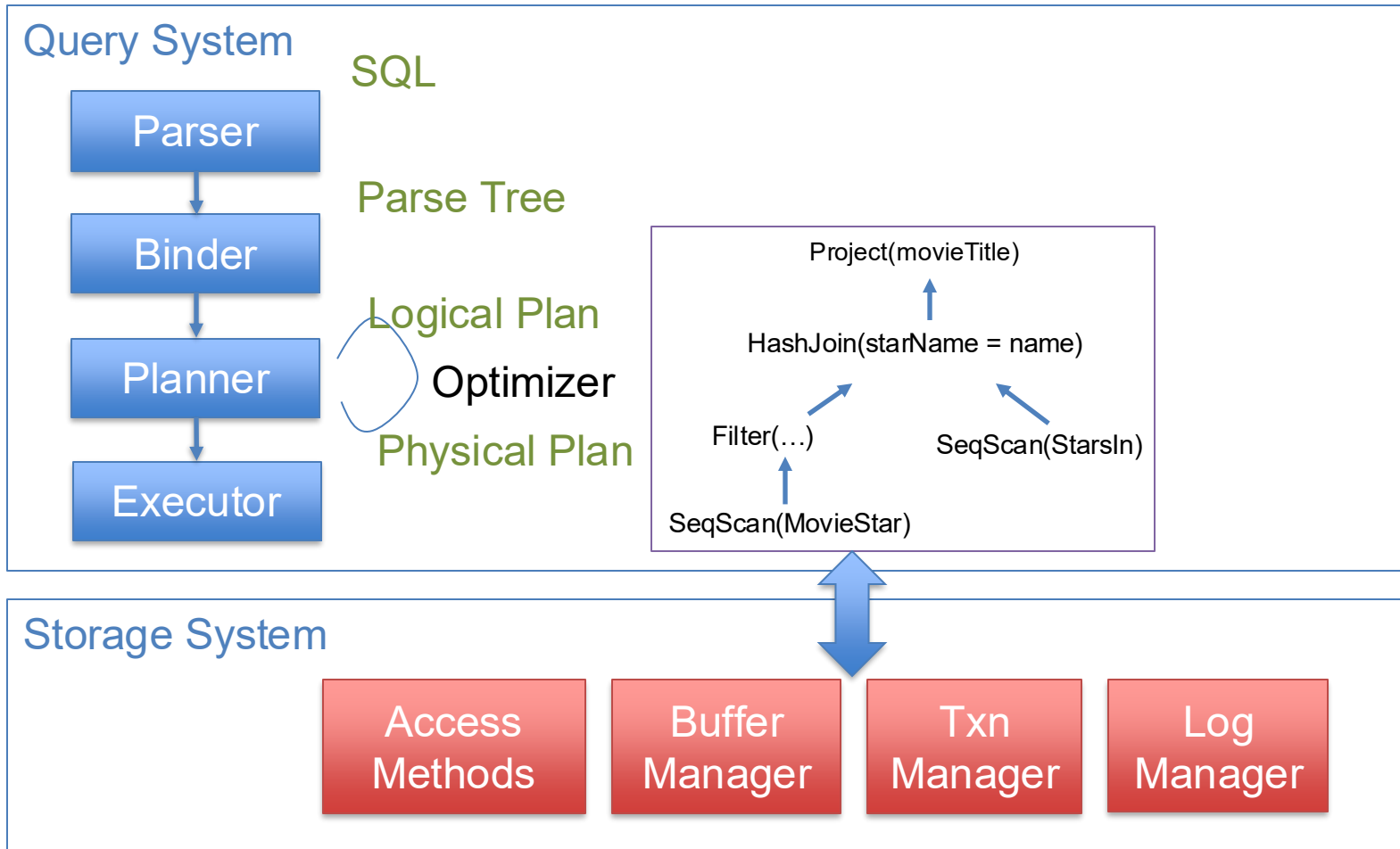
litianyu@mit.edu

Many thanks to Andy
Pavlo and CMU DB for
sharing their materials

Administrivia

- Pset 1 due today
- Lab 1 due next week
 - You really SHOULD get started!
 - Bootcamp recording available online
- Lab 2 out next week
 - Due 3/19
 - Significantly more code than lab 1, so start early and ask questions!

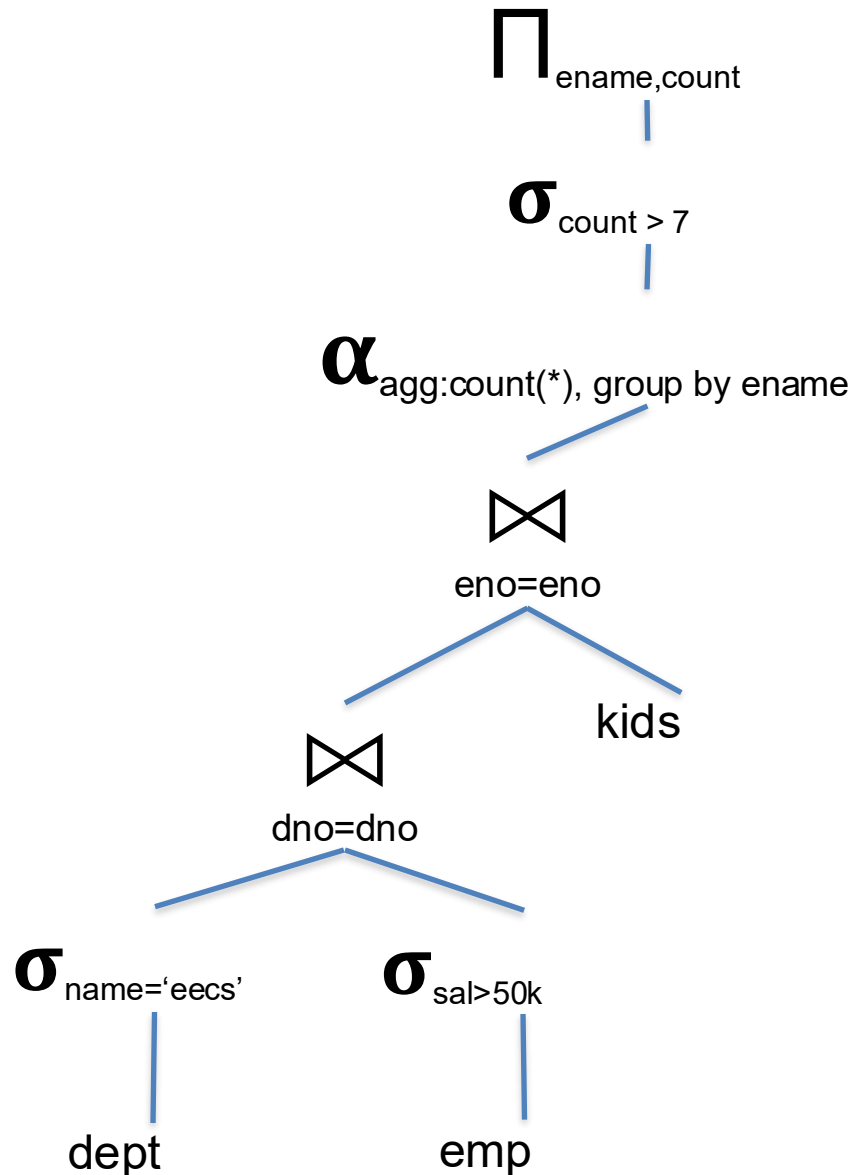
Recap



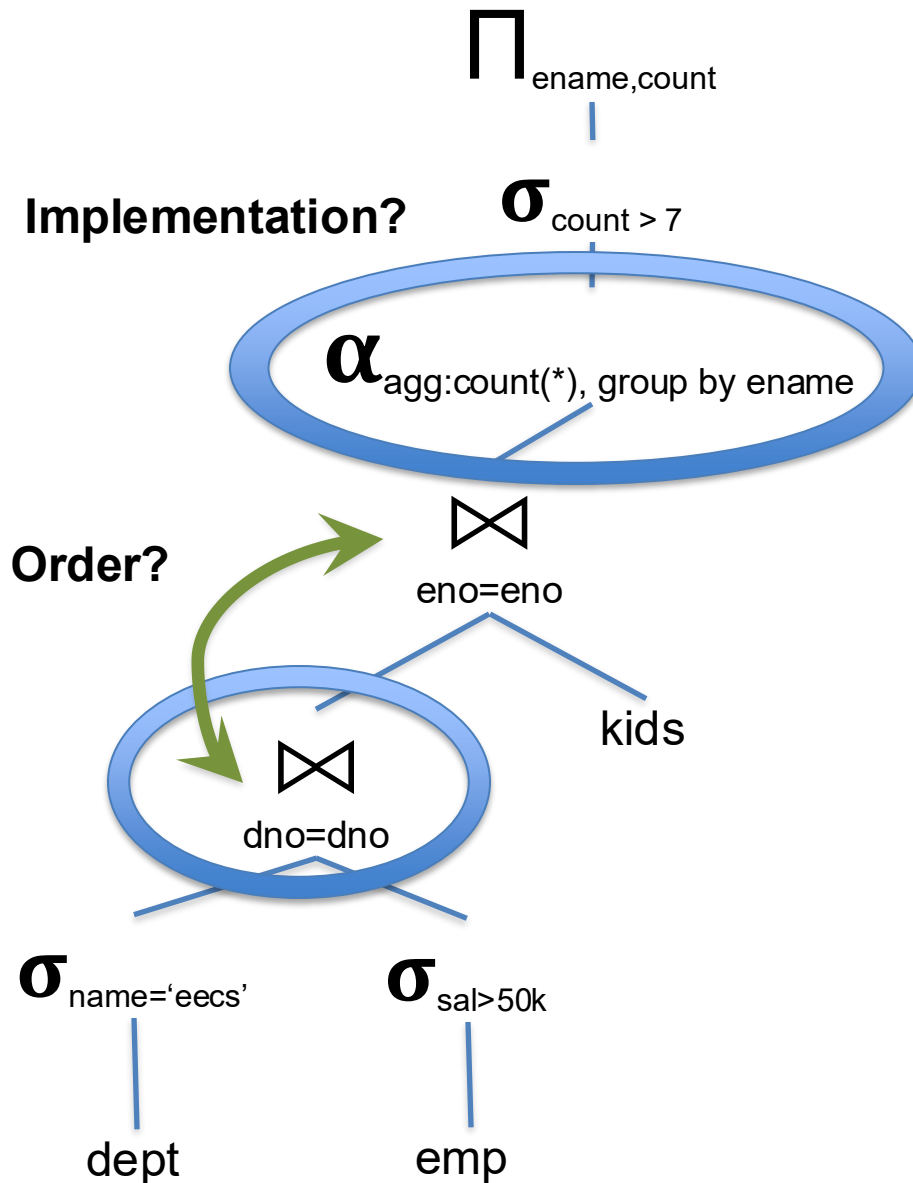
Recap: Query Plan

emp (eno, ename, sal, dno)
dept (dno, dname, bldg)
kids (kno, eno, kname, bday)

```
SELECT ename, count(*)  
FROM emp, dept, kids  
WHERE emp.dno=dept.dno  
AND kids.eno=emp.eno  
AND emp.sal > 50000  
AND dept.name = 'eecs'  
GROUP BY ename  
HAVING count(*) > 7
```



Recap: Query Optimization

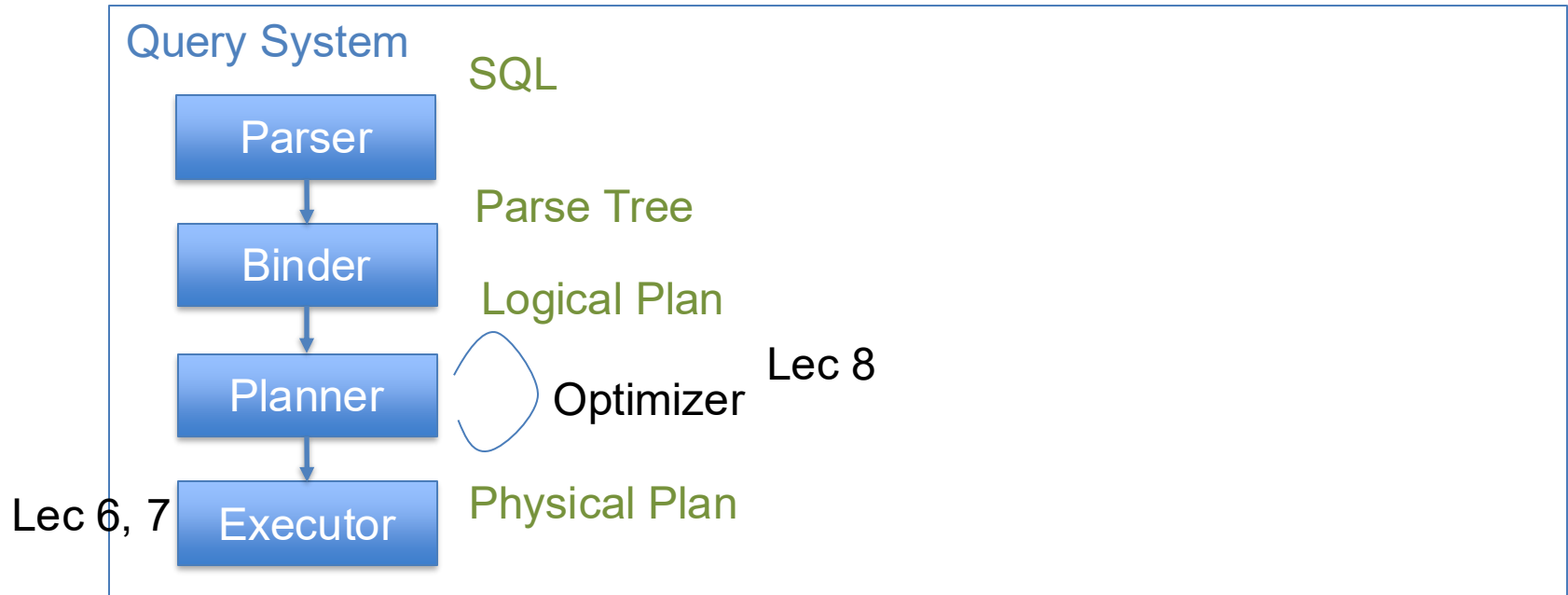


Logical planning:
operator ordering
(exponential search space)

Physical planning:
operator implementation
& access methods
(indexes vs heap files)

More on this in lec 8

Recap



Storage System

Lec 4

Today

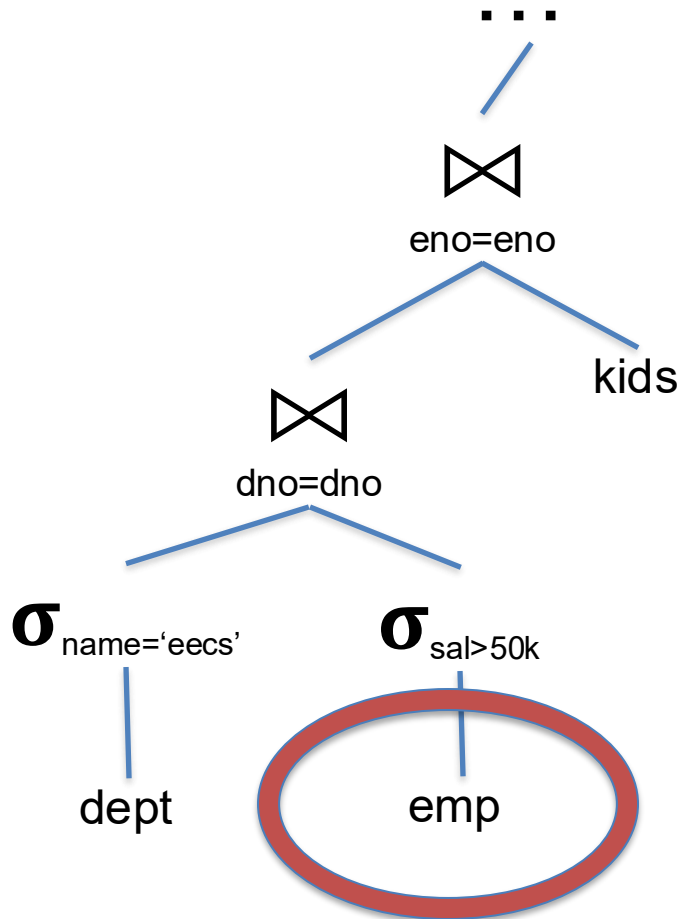
Access
Methods

Buffer
Manager

Txn
Manager

Log
Manager

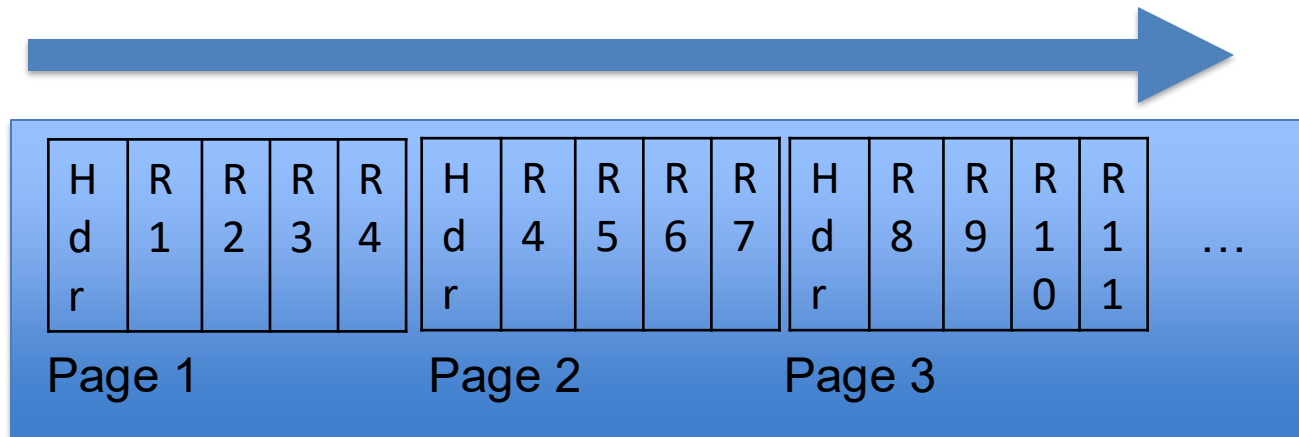
Today: Access Methods



- Access method: way to access the records of the database

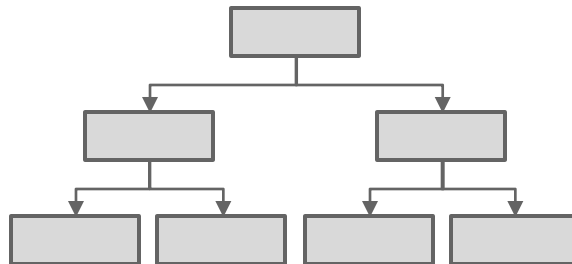
Heap Scan

- Read Heap File In Stored Order
 - Even with a predicate, read all records
 - Efficient if we indeed must scan all the records
 - Problematic otherwise



Indexes

- An **Index** maps from a key to record IDs
- Keys are constructed from one or more columns of the tuple
- Several types of indexes, including trees (most commonly B+Trees) and hash indexes



Index Access Methods

- Index Lookups (i.e., point queries)
 - Can be served from either index type
 - E.g., `SELECT * FROM emp WHERE id = 1`
- Index Scan (i.e., range queries)
 - Requires tree index
 - E.g., `SELECT * FROM emp WHERE sal > 10k AND sal < 100k`

Primary vs. Secondary Index

- Every table can have at most one primary index
 - Indexed key must be unique and non-null (i.e. primary key constraint)
 - Usually captures something about the underlying data model
- Tables can have additional “Secondary Indexes”
 - Duplicate/Nulls supported
 - Used to speed up operations

Clustered Index

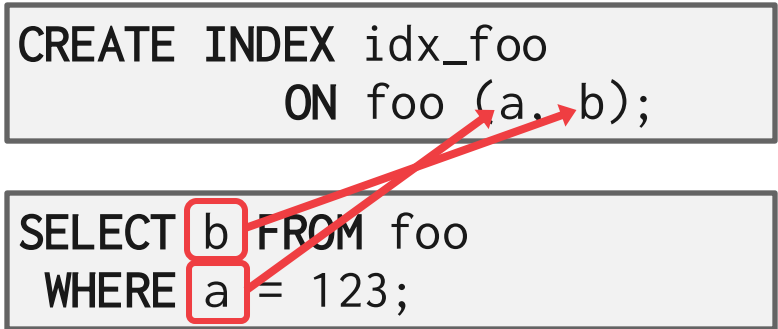
- The table is stored in the sort order specified by the indexed (usually primary) key
 - Either sort the heap file, or store data in leaf nodes of an on-disk index
- Some DBMSs always use a clustered index (e.g., MySQL)
 - Not every database support clustered index

Covering Index

- If all the fields needed to process the query are available in an index, then the DBMS does not need to retrieve the tuple.
- One can also instruct the DBMS to embed additional columns with INCLUDE
- Also called **index-only scans**.
 - However, will introduce concurrency issues if not careful

```
CREATE INDEX idx_foo  
ON foo (a, b);
```

```
SELECT b FROM foo  
WHERE a = 123;
```



Composite Index

- A **composite index** is when the key is comprised of two or more attributes.

- Example: Index on **<a,b,c>**

```
CREATE INDEX my_idx ON xxx (a, b DESC, c NULLS FIRST);
```

- DBMS can use a tree index if the query provides a “prefix” of composite key.

- Supported: **(a=1 AND b=2 AND c=3)**

- Supported: **(a=1 AND b=2)**

- Rarely Supported: **(b=2), (c=3)**

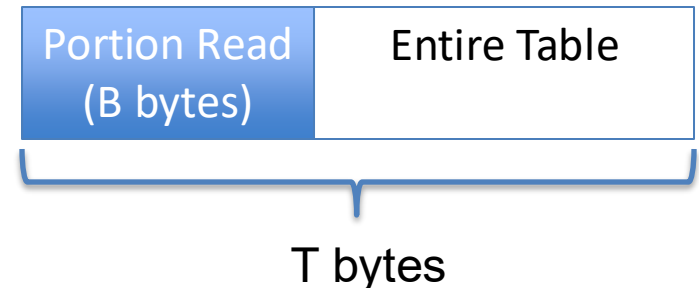
Index

- Compared to full table scans:
 - Efficient point lookup
 - Efficient range scans (tree indexes)
- Why not index everything?
 - Extra maintenance overhead on write
 - Extra space overhead
 - Often add indirection of access
 - Often introduce random access

Study Break

- Consider an SSD with 100 usec latency, 1 GB/sec BW
- Query accesses B bytes, R bytes per record, whole table is T bytes
- Seq scan time $S = T / 1\text{GB/sec}$
- Rand access via index time = $100 \text{ usec} * B/R + B / 1\text{GB/sec}$
- Suppose R is 100 bytes, T is 10 GB

For what data sizes is it cheaper to scan than do random lookups via index?



Study Break

- Consider an SSD with 100 usec latency, 1 GB/sec BW
- Query accesses B bytes, R bytes per record, whole table is T bytes
- Seq scan time $S = T / 1\text{GB/sec}$
- Rand access via index time = $100 \text{ usec} * B/R + B / 1\text{GB/sec}$
- Suppose R is 100 bytes, T is 10 GB

For what data sizes is it cheaper to scan than do random lookups via index?

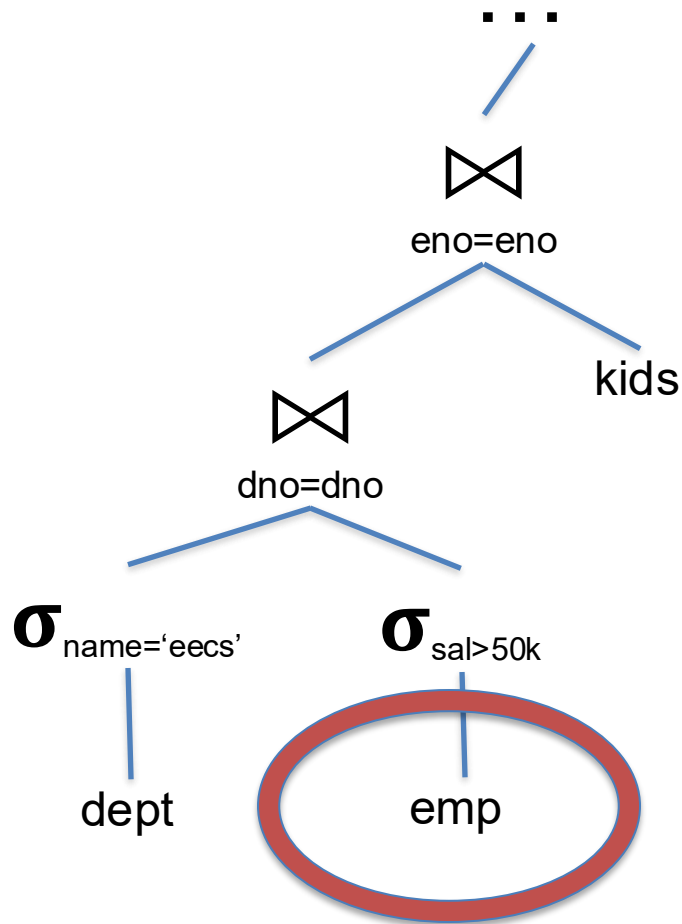
RANDOM > SCAN

$$100 \times 10^{-6} * B / 100 + B / 1 \times 10^9 > 10 \times 10^9 / 1 \times 10^9$$

$$1 \times 10^{-6} B + 1 \times 10^{-9} B > 10$$

$$B > 9.99 \times 10^6$$

For desired data sizes of larger than 10 MB, cheaper to scan entire 10 GB table than to use index



- Humans (or automated assistant) specify a physical design of the database (layout, index, etc.)
- Job of DBMS to pick the best access method based on cost

Access Method Costs

	Heap File	Hash File	B+Tree
Insert			
Delete			
Scan			
Lookup			

n : number of tuples

P : number of pages in file

Access Method Costs

	Heap File	Hash Index	B+Tree
Insert	$O(1)^*$		
Delete	$O(P)$ seq		
Scan	$O(P)$ seq		
Lookup	$O(P)$ seq		

n : number of tuples

P : number of pages in file

Hash Indexing: Idea

- Store a hash table with pointers to records in heap file
- Space Complexity: $O(n)$
- Time Complexity:
 - Average: $O(1)$  *Databases care about constants!*
 - Worst: $O(n)$
- Many possible implementations
 - We will not cover them in class
 - Usually a trade-off between speed and flexibility

Access Method Costs

	Heap File	Hash Index	B+Tree
Insert	$O(1)^*$		
Delete	$O(P)$ seq		
R-Scan	$O(P)$ seq		
Lookup	$O(P)$ seq		

n : number of tuples

P : number of pages in file

Access Method Costs

	Heap File	Hash Index	B+Tree
Insert	$O(1)^*$	$O(1)$ random	
Delete	$O(P)$ seq	$O(1)$ random	
R-Scan	$O(P)$ seq	- / $O(P)^*$	
Lookup	$O(P)$ seq	$O(1)$ random	

n : number of tuples

P : number of pages in file

B-Tree Indexes

- There is a specific data structure called a **B-Tree**.
- People also use the term to generally refer to a class of balanced tree data structures:
 - **B-Tree** (1970)
 - **B+Tree** (1973)
 - **B*Tree** (1977?)
 - **B^{link}-Tree** (1981)
 - **B&-Tree** (2003)
 - **Bw-Tree** (2013)
 - **Bf-Tree** (2024)

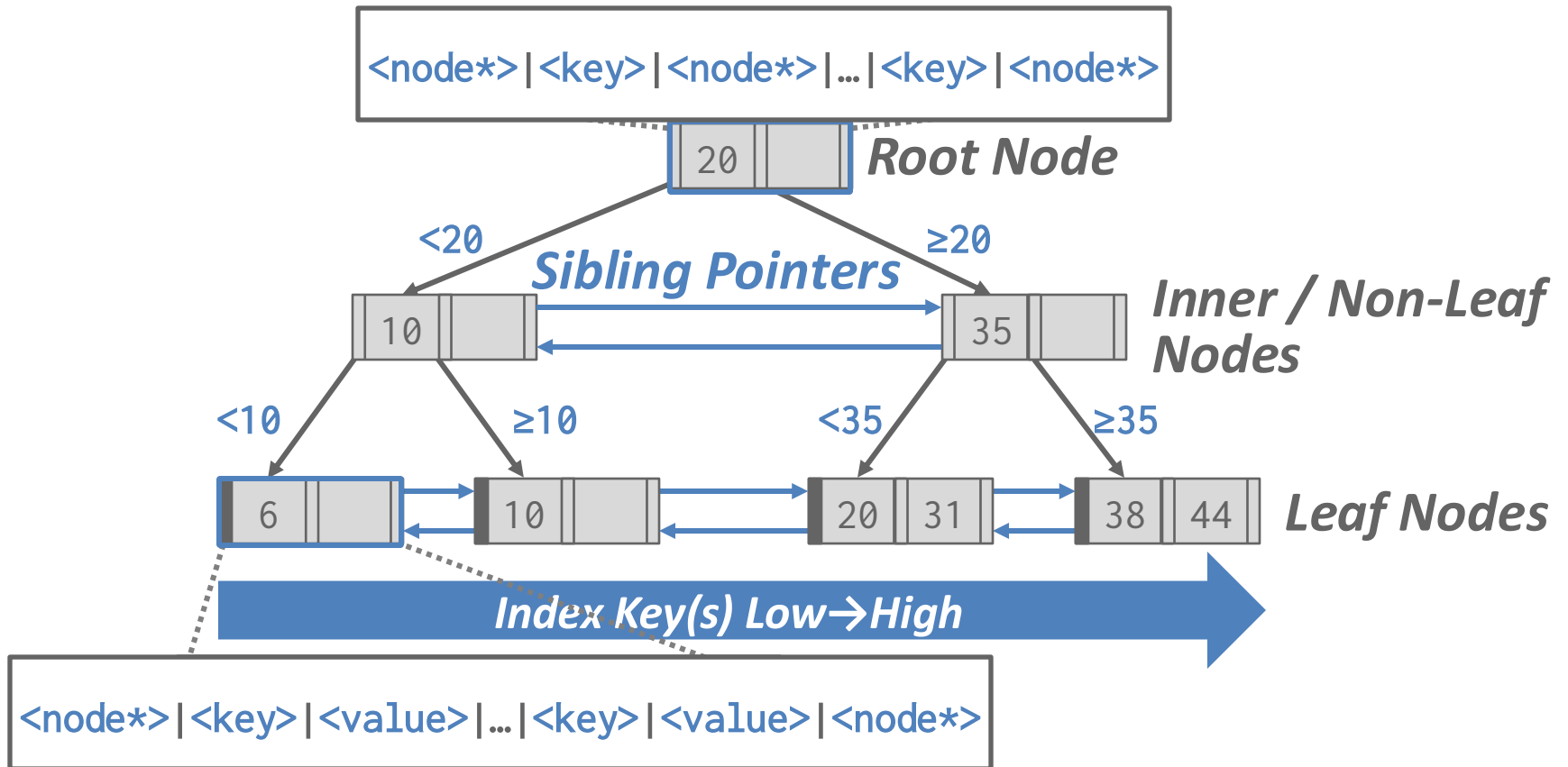
B+Tree

- A **B+Tree** is a self-balancing, ordered m -way tree for searches, sequential access, insertions, and deletions in $O(\log_m n)$ where m is the tree fanout.
 - It is perfectly balanced (i.e., every leaf node is at the same depth in the tree)
 - Every node other than the root is at least half-full
$$m/2 - 1 \leq \#keys \leq m - 1$$
 - Every inner node with k keys has $k+1$ non-null children.
 - Optimized for reading/writing large data blocks.
- Some real-world implementations relax these properties, but we will ignore that for now...

Nodes

- Every B+Tree node is comprised of an array of key/value pairs.
 - The keys are derived from the index's target attribute(s).
 - Duplicates are usually made unique
 - The values will differ based on whether the node is classified as an inner node or a leaf node.
- The arrays are (usually) kept in sorted key order.
- Leaf nodes either store record IDs or actual data

B+Tree Example

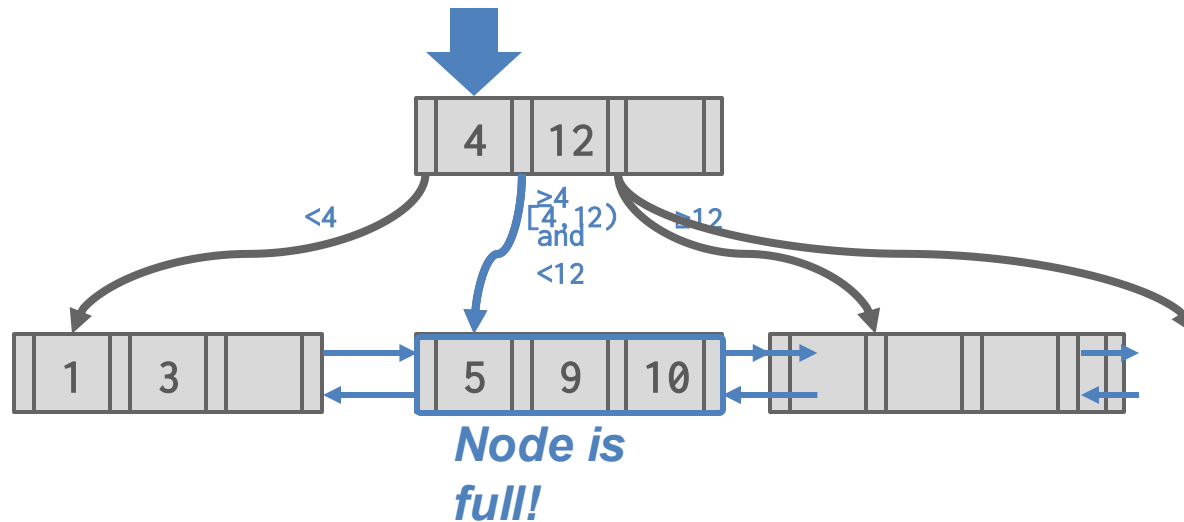


B+Tree: Insert

- Find correct leaf node L . Insert data entry into L in sorted order.
 - If L has enough space, done!
 - Otherwise, split L keys into L and a new node L_2
 - Redistribute entries evenly, copy up middle key.
 - Insert index entry pointing to L_2 into parent of L .
 - May require further splits

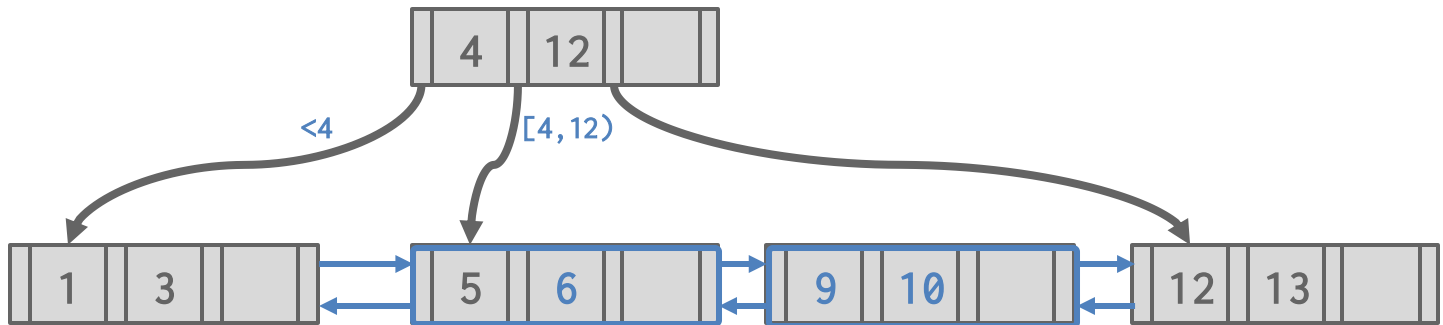
Example(1)

Insert 6



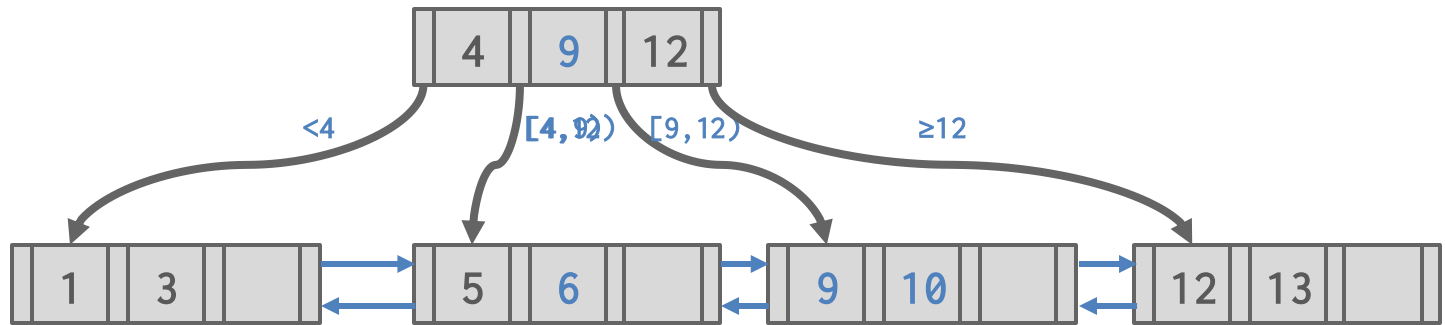
Example(1)

Insert 6



Example(1)

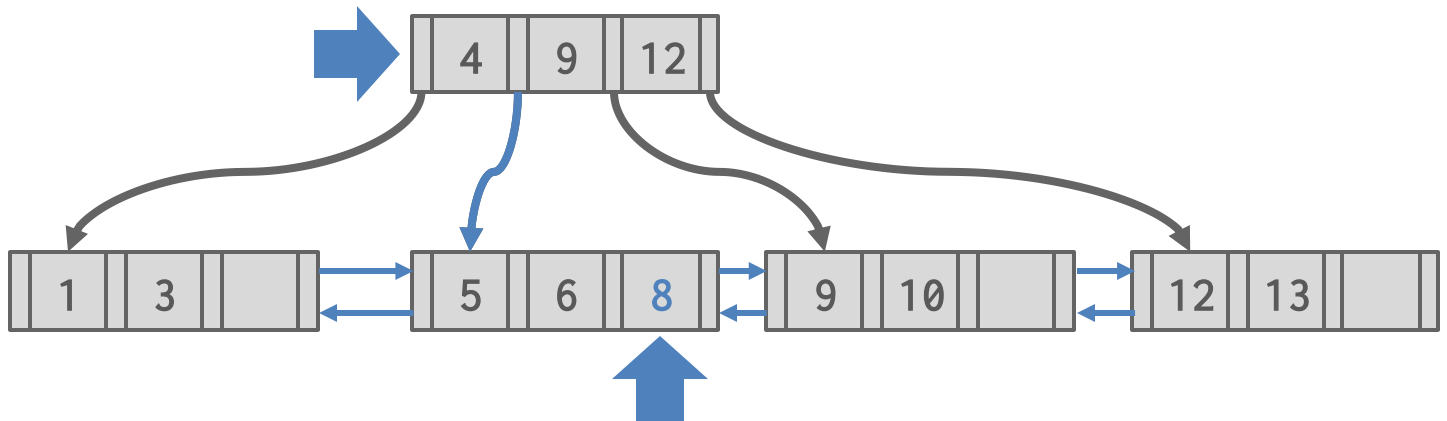
Insert 6



Example(1)

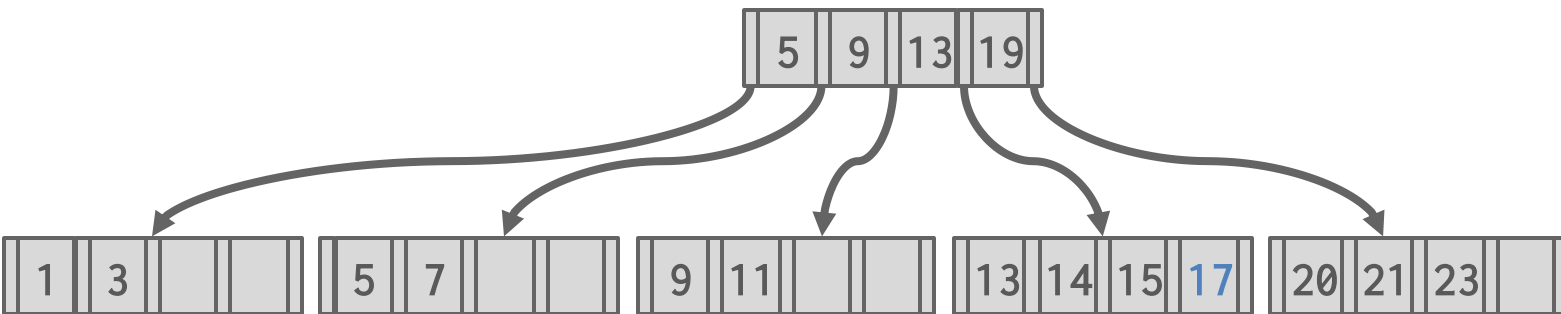
Insert 6

Insert 8



Example(2)

Insert 17

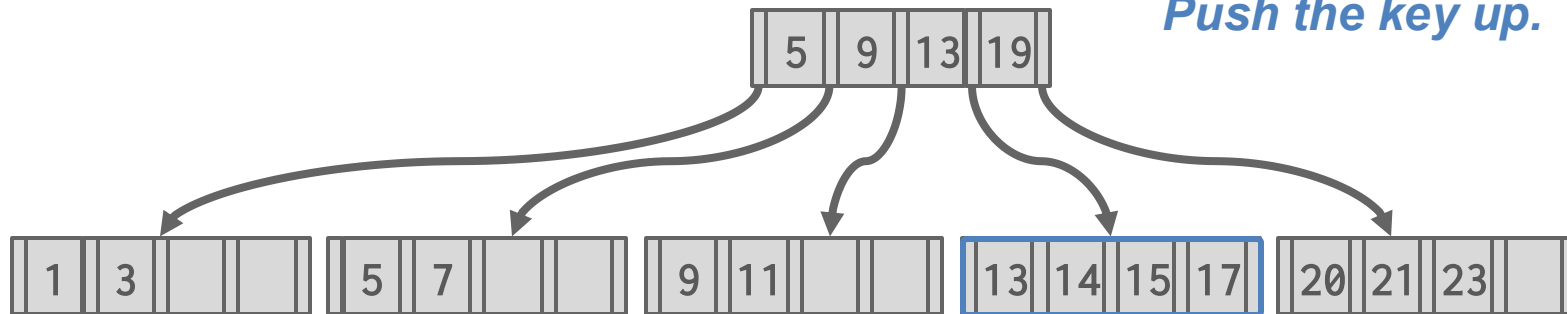


Example(2)

Insert 17

Insert 16

*Split the node!
Copy the middle
key.
Push the key up.*

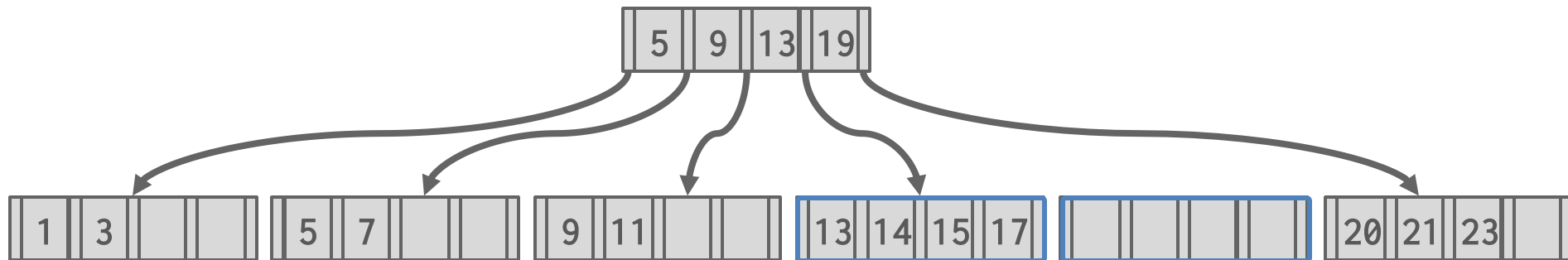


*No space in the node
where the new key
“belongs”.*

Example(2)

Insert 17

Insert 16



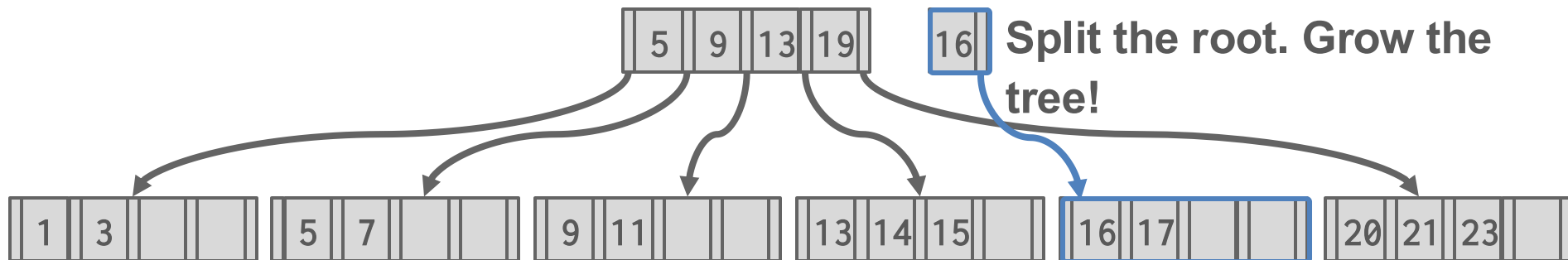
New Node!
Shuffle keys from the
node that triggered
the split.

Example(2)

Insert 17

Insert 16

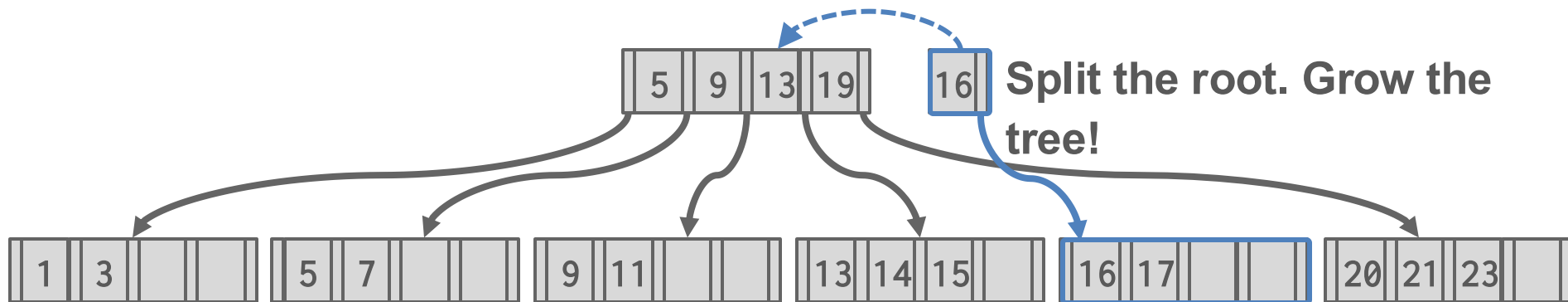
Want to create a key, pointer pair like this. But cannot insert it in the root node, which is full.



Example(2)

Insert 17

Insert 16

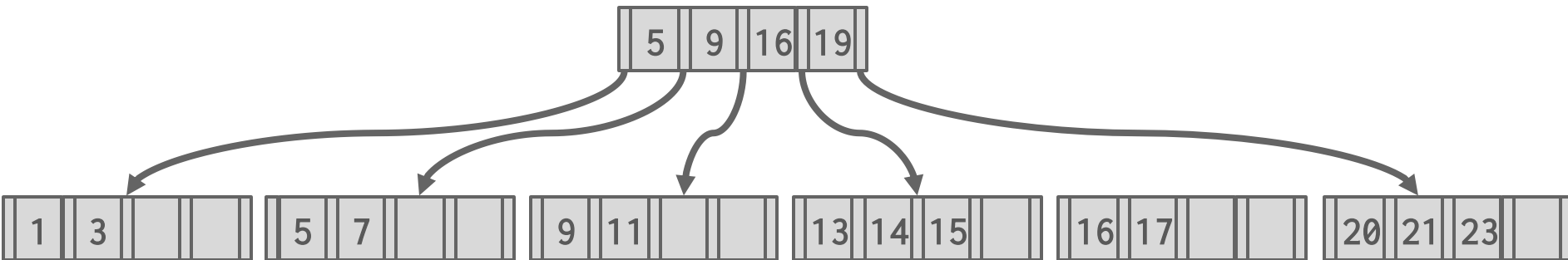


Example(2)

Next, need to split the “old” root, then point to the split nodes from the new root.

Insert 17

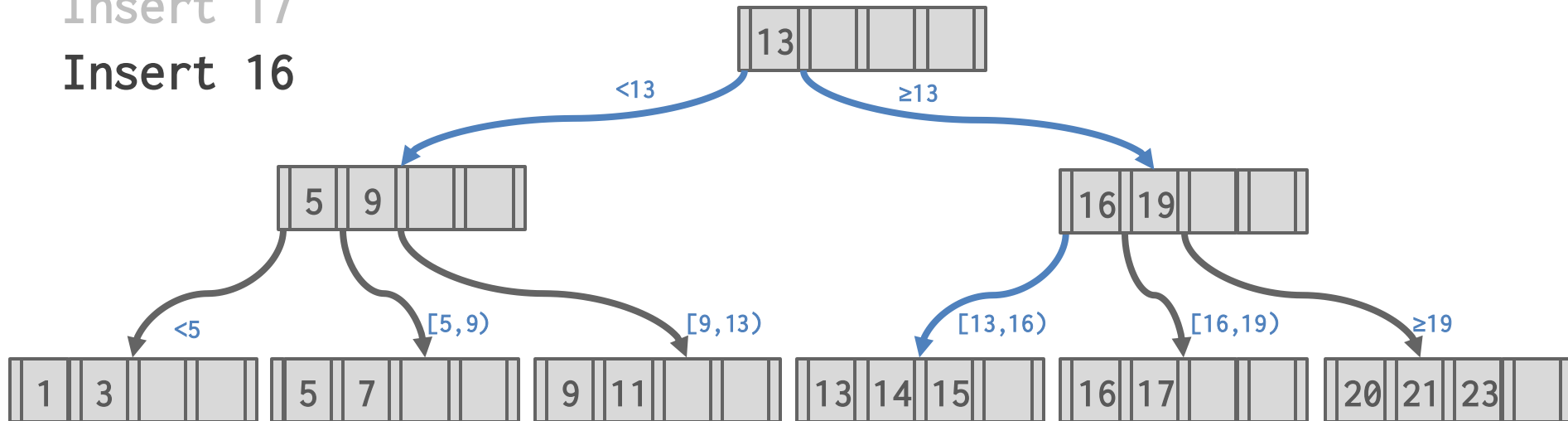
Insert 16



Example(2)

Insert 17

Insert 16

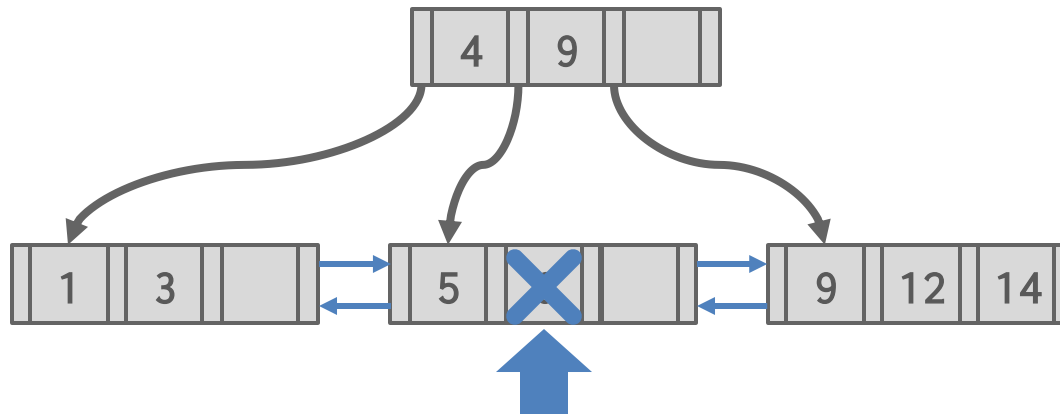


B+Tree: Delete

- Start at root, find leaf **L** where entry belongs.
Remove the entry.
If **L** is at least half-full, done!
If **L** has only $m/2-1$ entries,
 - Try to re-distribute, borrowing from sibling (adjacent node with same parent as **L**).
 - If re-distribution fails, merge **L** and sibling.
- If merge occurred, must delete entry (pointing to **L** or sibling) from parent of **L**.

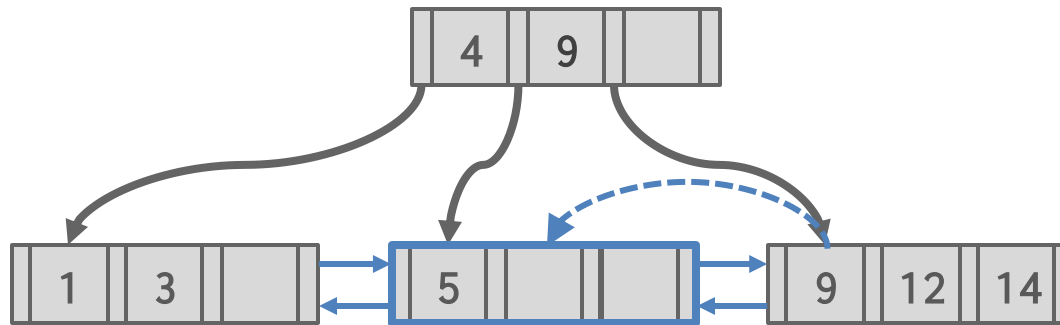
Example (3)

Delete 6



Example (3)

Delete 6

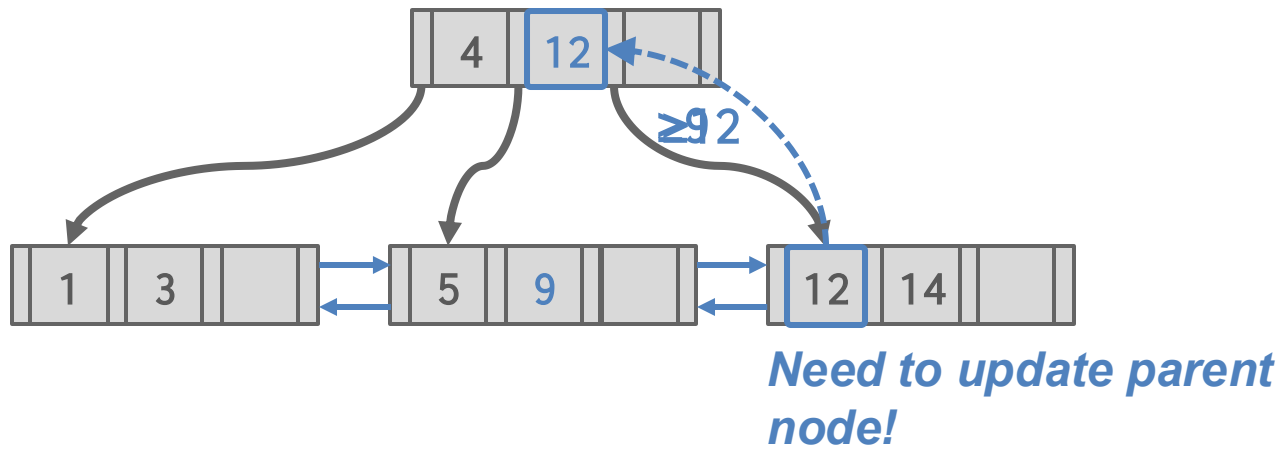


*Borrow from a “rich”
sibling node.*

*Could borrow from either
sibling.*

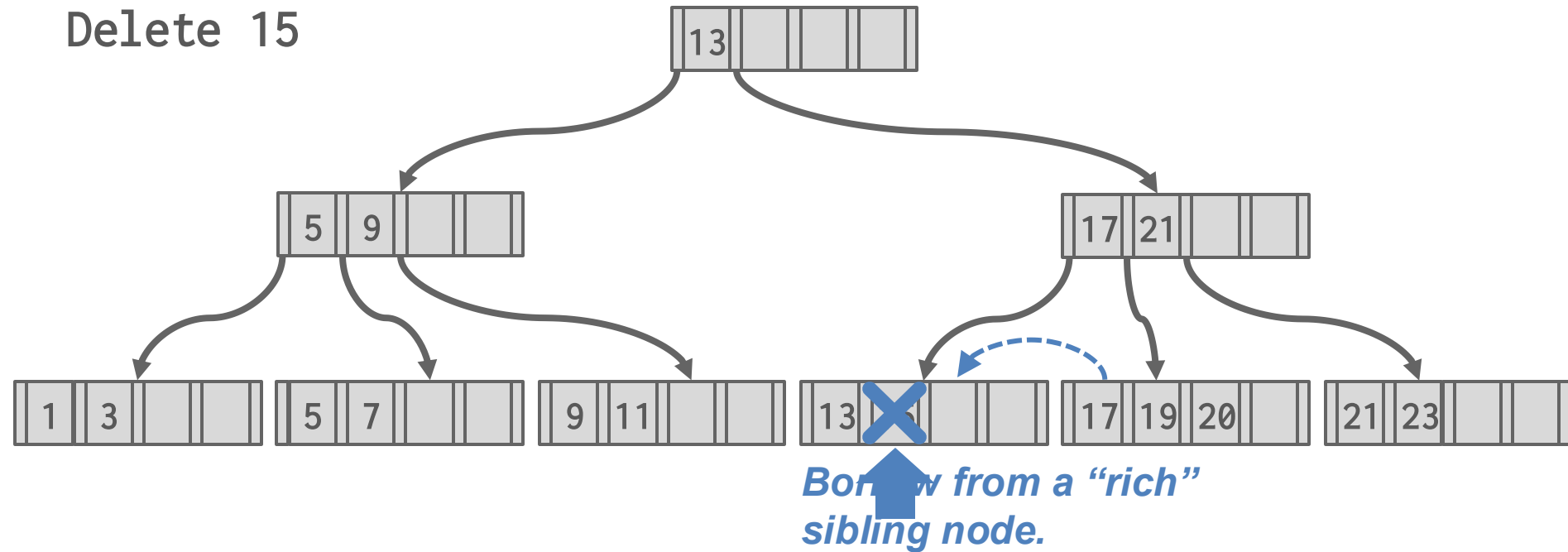
Example (3)

Delete 6



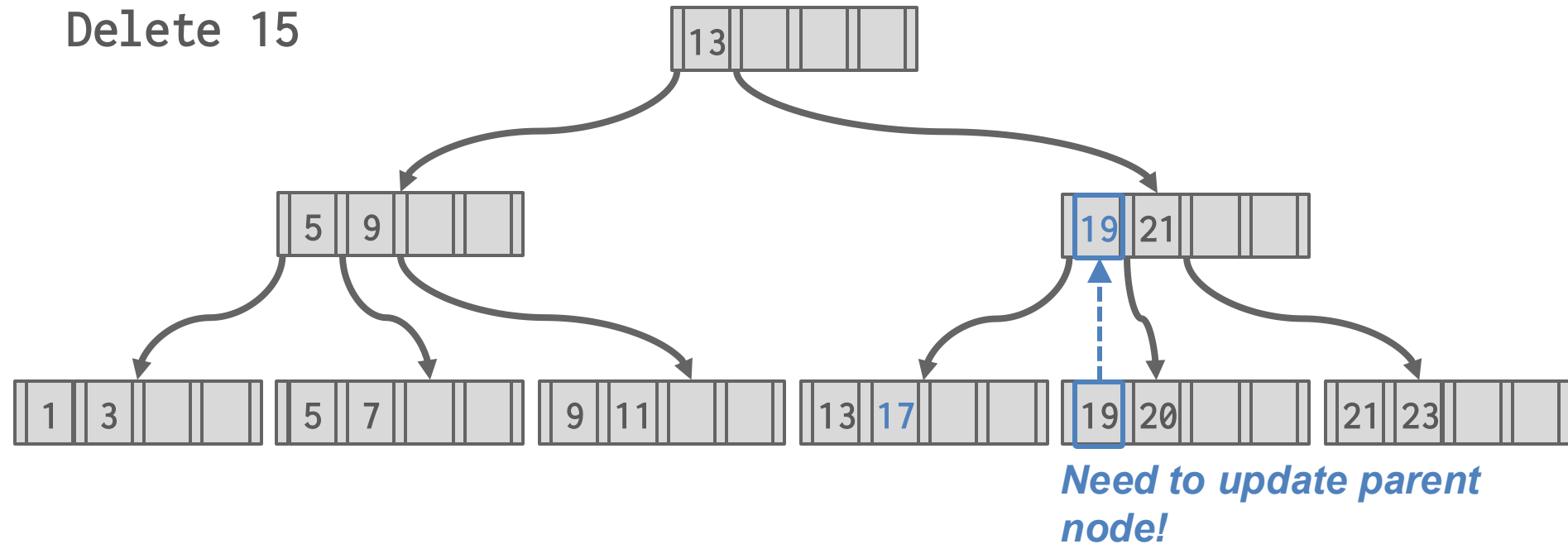
Example (4)

Delete 15



Example (4)

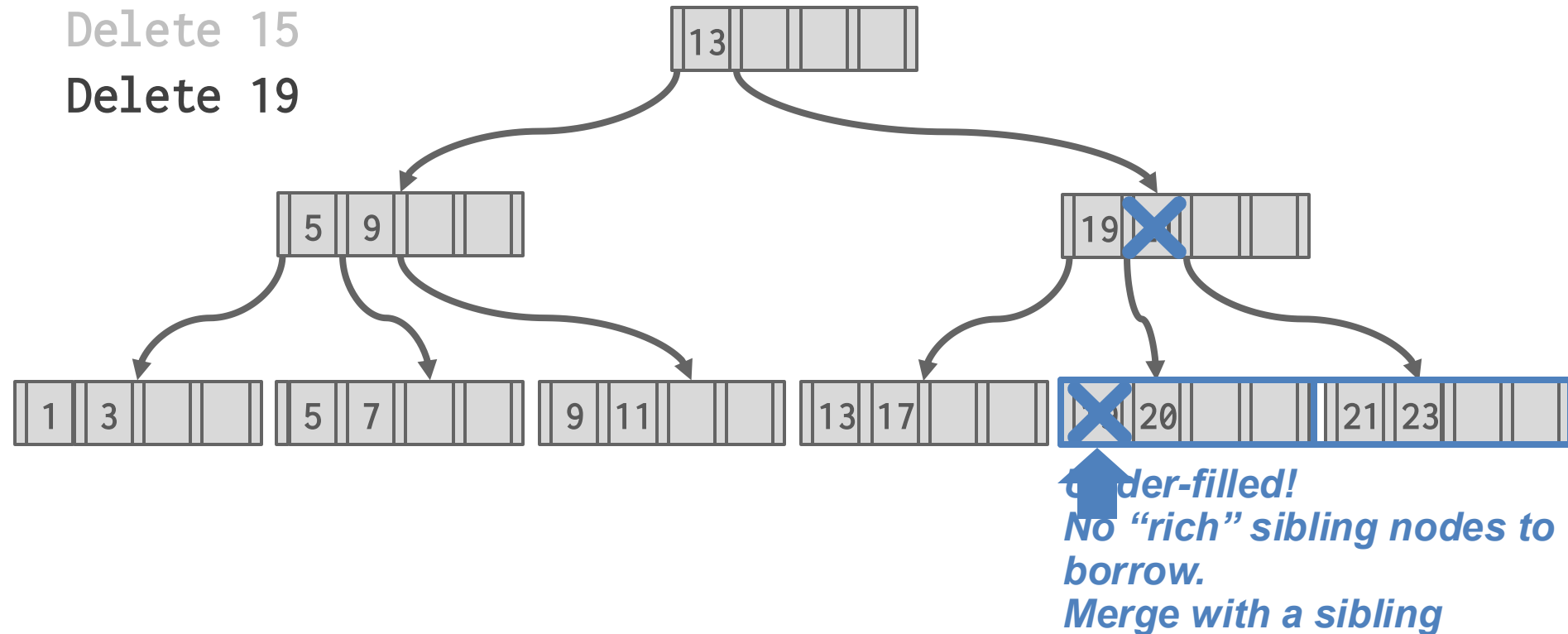
Delete 15



Example (4)

Delete 15

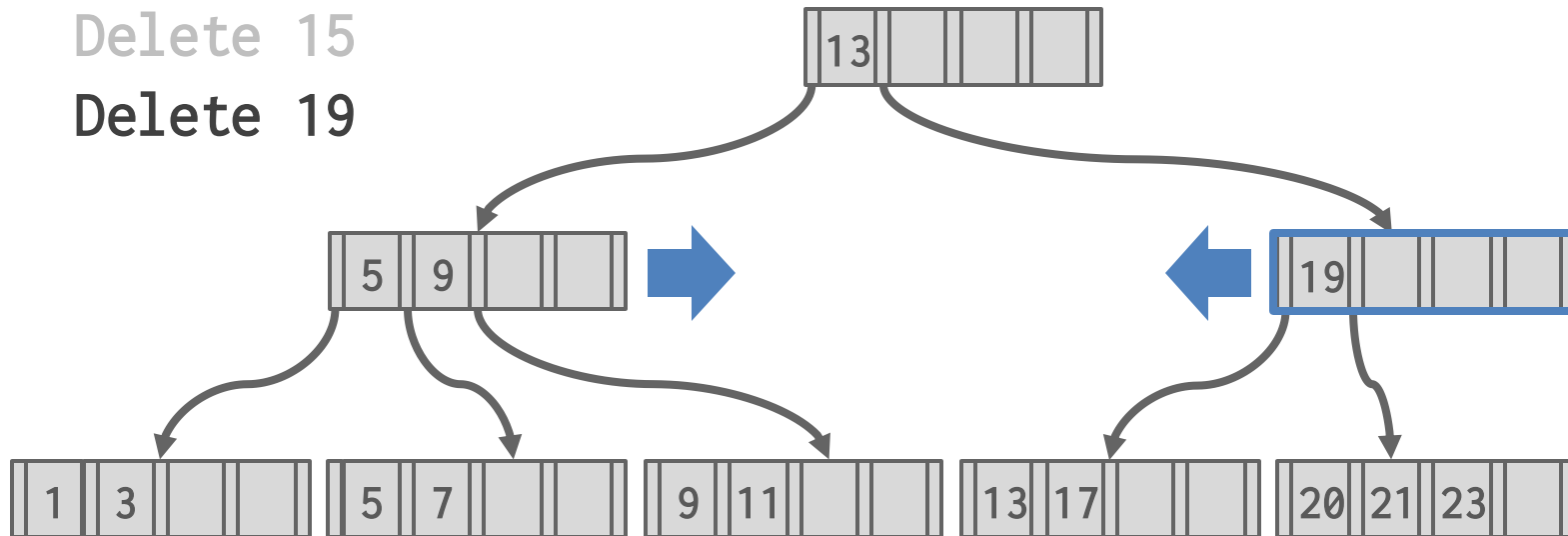
Delete 19



Example (4)

Delete 15

Delete 19



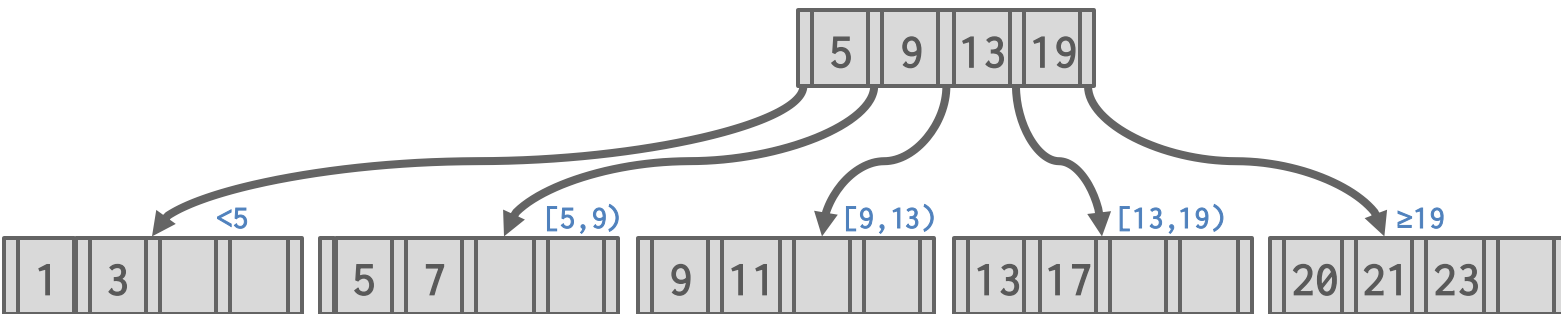
*This node
is
under-
filled!
Pull-down.*

Example (4)

Delete 15

Delete 19

The tree has shrunk in height.



B-Tree: Summary

- The venerable B+Tree is (almost) always a good choice for your DBMS.
 - More optimized for disk and scans compared to other self-balancing trees
 - Many variants
 - Many optimizations
- Unsurprisingly, this is often the “default” index used

Access Method Costs

	Heap File	Hash Index	B+Tree
Insert	$O(1)^*$	$O(1)$ random	
Delete	$O(P)$ seq	$O(1)$ random	
R-Scan	$O(P)$ seq	- / $O(P)^*$	
Lookup	$O(P)$ seq	$O(1)$ random	

n : number of tuples

P : number of pages in file

m : branching factor of B-Tree

R : number of pages in range

Access Method Costs

	Heap File	Hash Index	B+Tree
Insert	$O(1)^*$	$O(1)$ random	$O(\log_m n)$ random
Delete	$O(P)$ seq	$O(1)$ random	$O(\log_m n)$ random
R-Scan	$O(P)$ seq	- / $O(P)^*$	$O(\log_m n + R)$ seq*
Lookup	$O(P)$ seq	$O(1)$ random	$O(\log_m n)$ random

n : number of tuples

P : number of pages in file

m : branching factor of B-Tree

R : number of pages in range

Study Break

- What indexes would you create for the following queries (assuming emp table is very large, and consider each query in isolation)

`SELECT MAX(sal) FROM emp`

B+Tree on emp.sal (maybe)

`SELECT sal FROM emp WHERE id = 1`

Hash index on emp.id

`SELECT name FROM emp WHERE sal > 100k`

B+Tree on emp.sal (maybe)

`SELECT name FROM emp WHERE sal > 100k AND dept = 2`

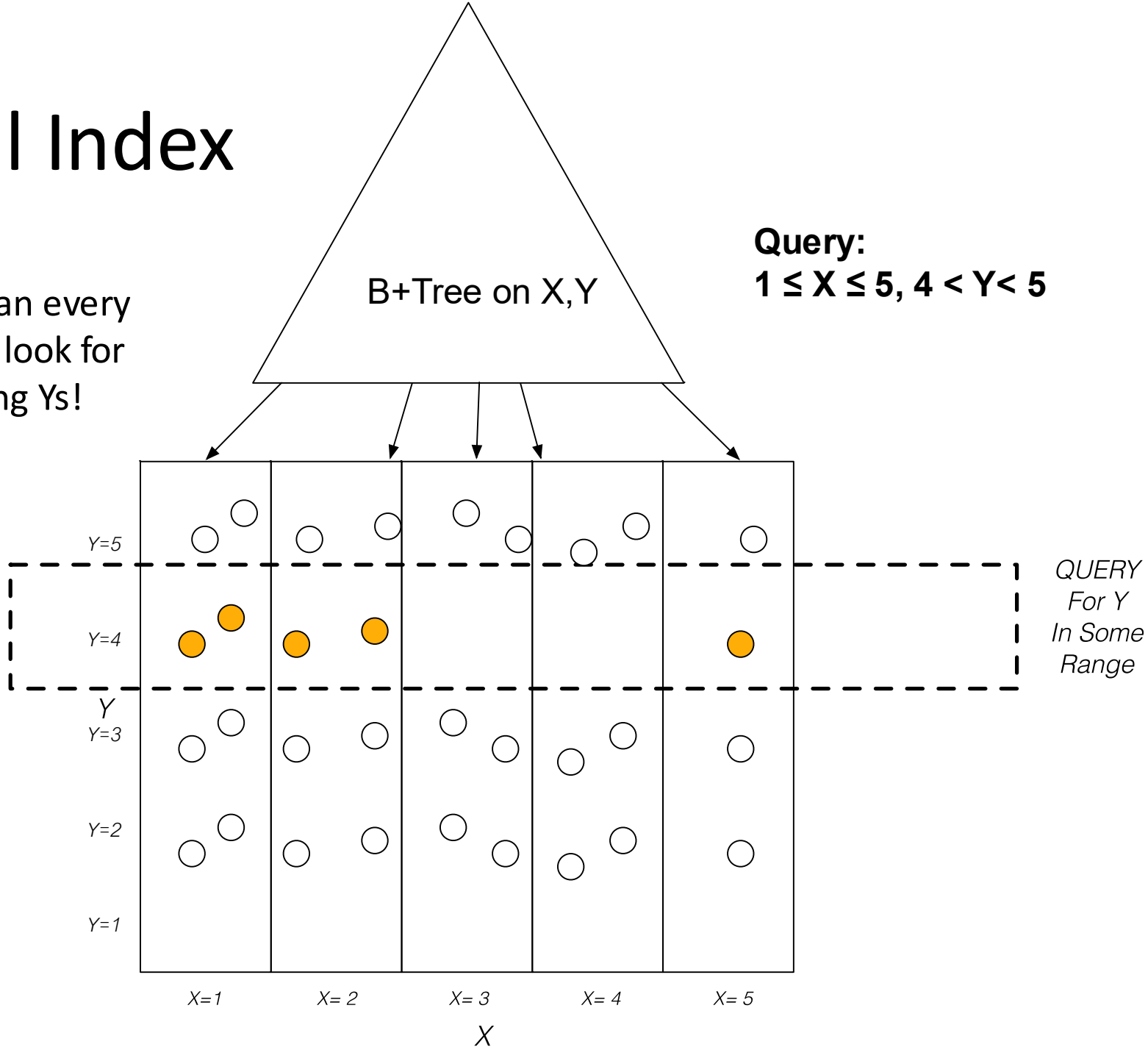
B+tree on emp.sal (maybe), Hash on dept.dno (maybe)

Automatic Index Design

- Problem Formulation: Given a query workload and a set of constraints, pick the optimal set of indexes
 - Unsurprisingly, NP-hard
- AutoAdmin from Microsoft Research (1996), and many others
 - Still area of active research!

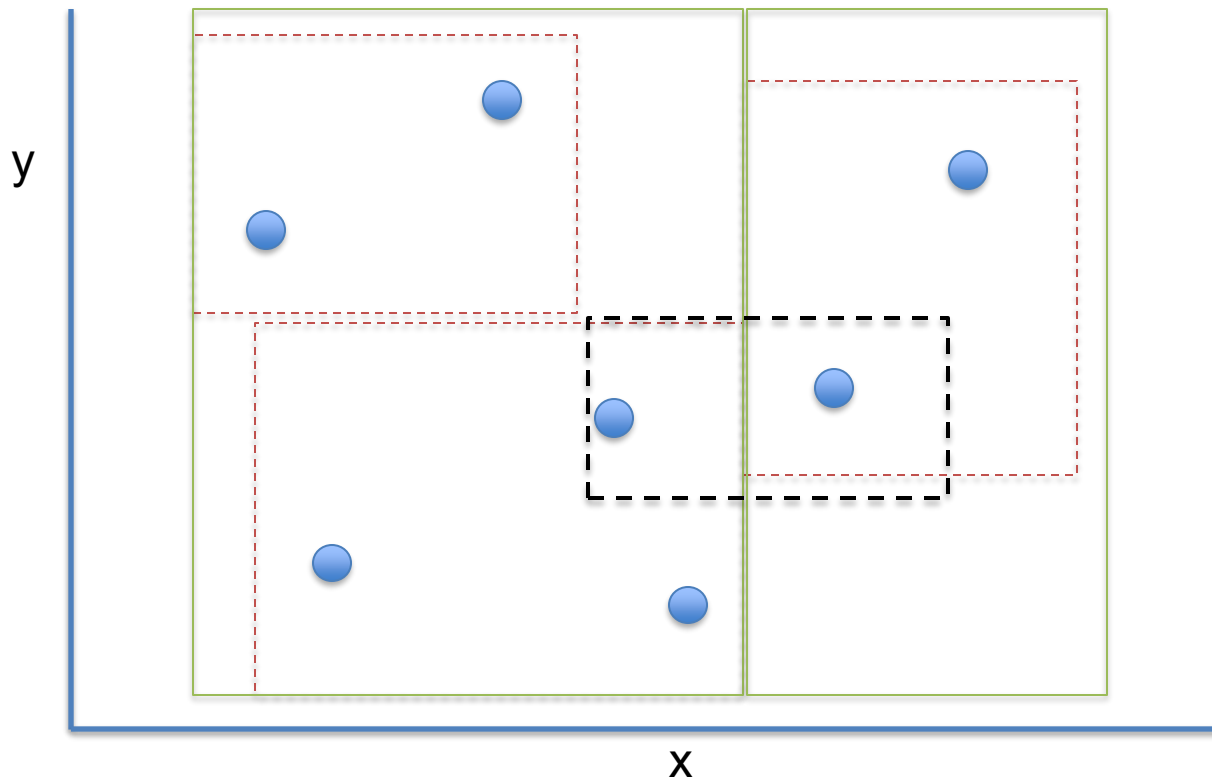
Spatial Index

Have to scan every
X value to look for
matching Ys!



R-Tree

- Generalize B-Tree to high dimensions
 - Nodes represent “bounding boxes”
 - Queries are in the form of rectangles

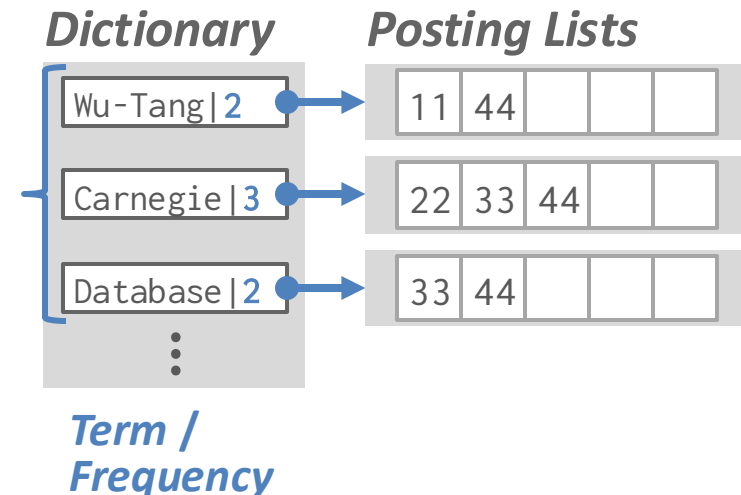


Inverted Indexes

- An **inverted index** stores a mapping of terms to records that contain those terms in the target attribute.
 - Sometimes called a ***full-text search index***.
- Many major DBMSs support these natively. But there are also specialized DBMSs and libraries.

revisions(id,content,...)

id	content
11	Wu-Tang Clan is an American hip hop musical collective formed in Staten Island, New York City, in 1992...
22	Carnegie Mellon University (CMU) is a private research university in Pittsburgh, Pennsylvania. The institution was established in 1900 by Andrew Carnegie...
33	In computing, a database is an organized collection of data or a type of data store based on the use of a database management system (DBMS), the software...
44	Sam Madden is MIT College of Computing Distinguished Professor in the EECS Department. He conducts research on database...



Observation

- Inverted indexes search data based on its contents with some term tweaking based on linguistic models.
 - Example: Normalization ("Wu-Tang" matches "Wu Tang").
- Instead of searching for records containing keywords (e.g., "Wu-Tang"), an application may want search for records that are semantically similar to topics (e.g., "hip-hop groups with songs about slinging").

Semantic Similarity Search

Album(id, name, year, lyrics)

id	name	year	lyrics
Id1	Enter the Wu-Tang	1993	<text>
Id2	Run the Jewels 2	2015	<text>
Id3	Liquid Swords	1995	<text>
Id4	We Got It from Here	2016	<text>



OpenAI



Hugging Face

Transformer

Embeddings

Id1 → [0.32, 0.78, 0.30, ...]
Id2 → [0.99, 0.19, 0.81, ...]
Id3 → [0.01, 0.18, 0.85, ...]
Id4 → [0.19, 0.82, 0.24, ...]
⋮

Query

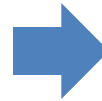
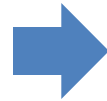
Find albums with lyrics about
running from the police and
released after **2005**

[0.02, 0.10, 0.24, ...]

Ranked List of Ids

year > 2005

*Vector
Index*



Vector Indexes

- Specialized data structures to perform nearest-neighbor searches on [embeddings](#) (i.e., one-dimensional arrays of floating-point numbers).
 - May also need to filter data before / after vector searches.
 - Result correctness depends on whether the result "feels right".
- More on this in Lecture 21

Next Class

- Query Execution + Operators
 - You will implement these in Lab 2
- Get started on Lab 1!